

Ontology-based Requirements Transformation

Jan Novacek^{*†}, Alexander Viehl^{*}, Oliver Bringmann^{*†}, Wolfgang Rosenstiel^{*†}

^{*}FZI Forschungszentrum Informatik
Haid-und-Neu-Str. 10-14, 76131 Karlsruhe

[†]Eberhard Karls Universität Tübingen
Sand 14, 72076 Tübingen

Abstract—This paper presents an ontology-based approach to the supply chain-aware transformation of functional and environmental load requirements given by so-called Mission Profiles (MPs). The approach aims at improving the efficiency of the engineering process through supporting the transformation process and enabling a better integration of the transformation into existing Model-based Systems Engineering (MBSE) processes. We propose a methodology and a supporting system which aids in the transformation process while the latter feature is obtained by constructing and working on models. Consequent utilization of the standardized language OWL to express model representations further enables better knowledge integration and transfer among heterogeneous systems. In addition to that, this favors knowledge reuse across projects which can reduce overall costs. Moreover, the system enables stripping off irrelevant information from MPs, thus improving protection of intellectual property. **Index Terms**—Systems Engineering, Knowledge-Based Engineering, Requirements Management, Mission Profiles, Model-to-Model Transformation

I. INTRODUCTION

One important aspect of the tremendous increase of system complexity is the resulting necessity to manage huge amounts of requirements efficiently. This includes the processing of requirements for transforming them as they are broken down to separate tier needs. The data transformation framework presented in this paper aims specifically at improving the efficiency of the engineering process through supporting the process of transforming functional and environmental load requirements given by so-called Mission Profiles (MPs). Ontologies are used to capture knowledge and to represent models to enable a better integration of heterogeneous data sources and to encourage reuse of knowledge across project life cycle phases and between projects which has been identified as an important aspect of today's Systems Engineering challenges [1].

Typically, requirements are passed along the supply chain while being refined and broken down to subsequent tier needs. In this process MP data needs to be transformed, see Fig. 1. The selection of an appropriate transformation, its parametrization, and applying it to the MP data can be supported by using ontologies and reasoning. Using MBSE methodologies is accepted as an effective way in the design process of complex systems in the automotive and aerospace industries [1, 2]. The presented approach follows the idea of using and expressing models through the standardized ontology language OWL [3] and the Semantic Web Rule Language (SWRL) [4] to further improve knowledge sharing and reuse among heterogeneous systems. Therefore, the approach can be integrated into existing MBSE methods. It supports multiple Model-to-Model (M2M) transformation approaches and is extensible. Basically, this paper aims at providing support for the process of transforming MPs. We investigate on whether and if so how Semantic Web Technologies (SWTs) can be used to accomplish this, focusing on the use of ontologies.

The design of complex systems imposes several engineering challenges. A predominant problem is that Systems Engineering data is primarily stored in documents that are scattered across engineering workstations, and poorly managed [1]. With regard to requirements,

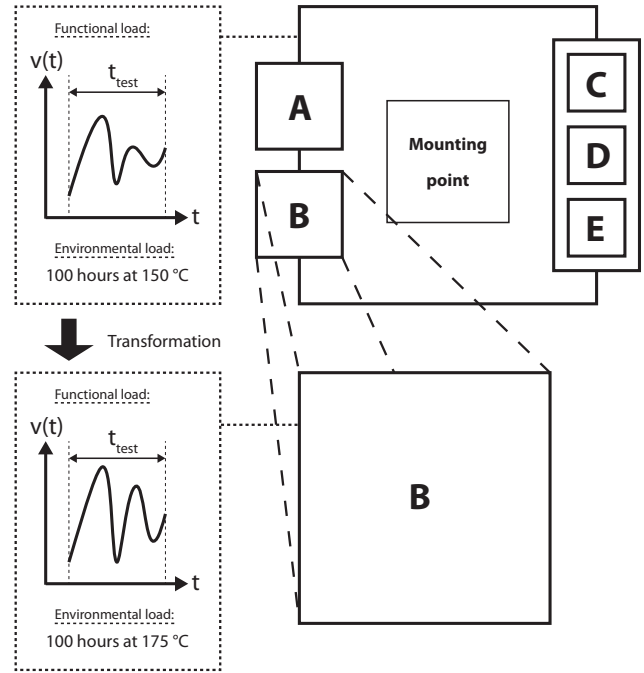


Fig. 1: MP transformation is necessary due to the difference in functional and environmental loads of port B of a component with a single mounting point

MPs are shared in a non-standardized way for example by using Excel sheets or Word documents. Transforming them automatically is therefore challenging as it requires implementing a custom solution for each MP representation. Another problem is the consideration of system information and the integration of relevant data into the transformation process.

This paper addresses the aforementioned problems through the proposal of a system which is capable of integrating heterogeneous data sources to build up models. The main goal of supporting MP transformation is then carried out using the constructed models. The benefits are primarily:

- Better protection of intellectual property (IP) due to removal of non necessary information
- Decreased communication overhead due to less transformations
- Better sharing and reuse of transformations across projects
- Avoidance of errors in the transformation process
- Reduced costs due to decreased complexity

As all models used in the presented approach are ontology-based, we follow the principle of Linked Data that allows interlinking and discovery of new data sources in a global data graph [5]. Relying

on an established standard for an ontology language, the system also favors sharing and processing of system models used in development. Creating effective Knowledge Management Systems is a key success factor in engineering process improvement [6].

This paper is divided into six sections. This Introduction is followed by a section providing relevant information on the background and related work. Section III describes requirements of a MP transformation system. In section IV the solution approach is presented. Section V describes two use cases. Finally, a conclusion drawn from the experiences made while carrying out this research is given in section VI.

II. BACKGROUND AND RELATED WORK

This section provides relevant background knowledge. The concepts of MPs and Mission Profile Aware Design (MPAD) are introduced and related work is mentioned. Also, the relation of works from the field of M2M transformation to our approach is described.

A. Mission Profiles

Capturing and specification of environmental conditions and functional loads which components are exposed to throughout their life cycle is done via MPs [7]. Therefore, they play an important role in the design and development of automotive systems, especially with regard to the propagation of requirements along the development process chain [8, 9]. To create a MP various sources are used such as load usage and environmental profiles, requirements and scenarios for stress tests. An initial draft of a format for MPs was developed in the RESCAR2.0 project¹ and is further described in [8]. To improve the collaboration among partners involved in automotive component design the autoSWIFT project² aimed to prepare a standard for MPs, the Mission Profile Format (MPFO)³.

B. Mission Profile Aware Design

Jerke and Kahng introduced the general concept of MPAD in [10]. They emphasized that MP consideration is still mainly a manual task. This is why it is important to provide tool support to enable automation and ease adoption. Currently, we are only aware of a single framework supporting MPAD, the *Reliability Knowledge Framework* by NXP which was created to connect users, reliability knowledge, data, tools and methods [11]. The rareness of software supporting MPAD might be due to the fact that a MP standard does not exist yet, see section II-A.

C. Model-to-Model transformation

While M2M transformation approaches can be divided into relational/declarative, imperative/operational, graph-based and hybrid categories [12] the approach to MP transformation presented in this paper supports multiple M2M transformation approaches and is not limited to one category. This is due to an identified key requirement, which is explained in section III-A.

III. REQUIREMENTS

In this section we list hypotheses from which we draw vital conclusions in form of identified abstract key requirements. Based on the key requirements, we also define evaluation criteria. This gradual refinement reflects our procedure.

¹<https://www.edacentrum.de/rescar/>

²<https://www.edacentrum.de/autoswift/>

³<http://www.mpfo.org/>

A. Key requirements

There are three important hypotheses which we used as a starting point for our considerations.

1) *Supply chain consideration*: At various tiers different portions of the original MP data are required. To apply failure models at the technology provider level for example requires maintenance of specific MP parameters at preceding tiers, see Fig. 3. This can also occur on higher levels. Tools for MP transformation should therefore be aware of the supply chain.

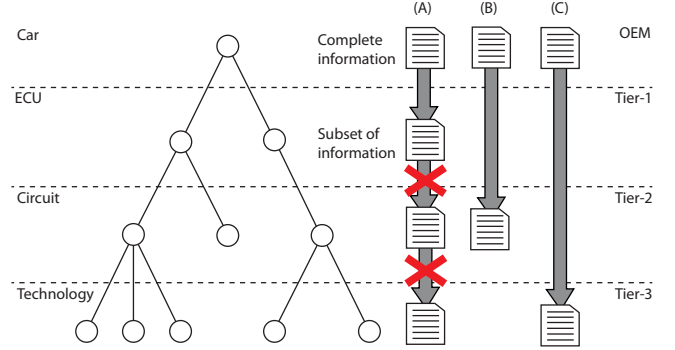


Fig. 3: Exemplary supply chain and MP propagation. At the OEM level all information is available as MP data. Transformation of the base MP for the Tier-1 typically results in a subset of MP information. Subsequent transfer of MP data from Tier-1 to Tier-2 is not possible as there might be information missing due to the initial transformation (A). Therefore, a cumbersome process involving transformations of the base MP to each tier's demands is necessary (B), (C)

2) *Generic load specifications*: For providing a general solution to the specification of environmental loads the means to describe the loads have to be versatile. This also applies to the feature set of tools used to process these load specifications. Discovery of previously unknown loads which have an impact on the component in question demands for extensibility of load specification mechanisms and related manipulation functionality of processing tools.

3) *Varying level of detail*: The level of detail of MPs can vary. Therefore tools for processing MP data need to be scalable. In addition to that, tools for processing MP data should be adaptable in terms of customizable manipulation operations to handle different levels of detail equally well.

We extracted the following abstract key requirements for a MP transformation system from the hypotheses:

- ① Maintenance mechanism for parameters
- ② Versatile processing feature set
- ③ Extensible processing feature set
- ④ Customizable manipulation operations
- ⑤ Scalability

B. Evaluation criteria

The level of abstraction of the former requirements makes it hard to decide what measures are suitable to meet a requirement because it is hard to tell whether it has been fulfilled or not. This also hinders a proper evaluation of the solution later on. To tackle these issues, we defined specific evaluation criteria [EC] associated with the key requirements which are more precise and specific:

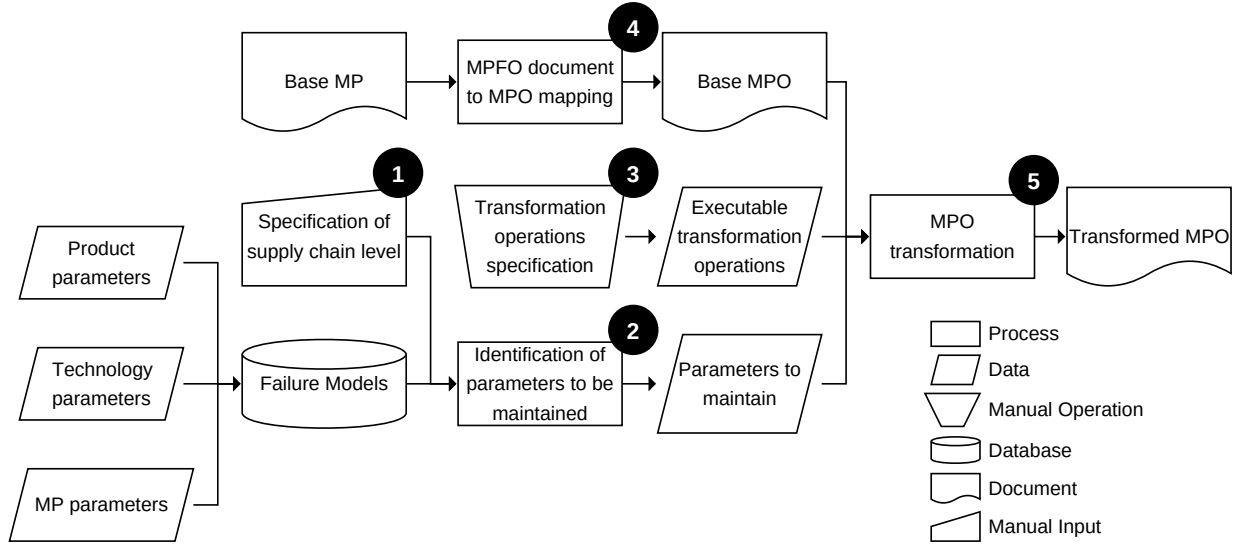


Fig. 2: Overview of the approach

EC.Parameter	The transformation system contains a mechanism for maintaining parameters required for subsequent tiers
EC.Transform	The transformation system contains at least two transformation approaches which belong to different classes of transformations according to the literature
EC.Extension	The transformation system provides a mechanism to extend it with new transformation approaches
EC.Modify	The transformation behavior of transformation approaches provided by the system is modifiable
EC.Detail	The transformation system should handle different detail levels of MP data equally well

IV. APPROACH

This section describes our proposal for ontology-based MP transformation. A first section providing an overview of the approach follows a description of the solution concept and a section with a brief summary of the design of a framework which was used to carry out this research. Finally, there is a section describing the architecture of the proposed system.

A. Overview

Please refer to Fig. 2 for an overview of the approach. The tier needs to be manually specified ❶. This information is then used in conjunction with known failure models for the identification of parameters to be maintained ❷ through the transformation. The manual specification of transformation operations yields executable transformation operations ❸. The MPFO base MP document is mapped to an MP ontology (MPO) using the framework described in section IV-C ❹. Finally, the ontology-based transformation of the base MPO is executed ❺.

B. Solution concept

A fundamental characteristic of our proposed solution is continuous use of a unifying language to express models namely the ontology language OWL. The primary purpose of this is to alleviate difficulties and effort to integrate data, information and knowledge required to fulfill the task of MP transformation and support its implementation.

In order to obtain models expressed in OWL we suggest *lifting* existing models, by which we mean a) translation of the source model into suitable OWL expressions and b) enrichment of the resulting ontology with additional relevant knowledge to support task achievement. Please refer to section IV-C for a description of the lifting mechanism we used in this approach to map MPs to ontologies.

The following five paragraphs refer to the previously defined evaluation criteria and outline the measures undertaken to satisfy the key requirements and to achieve the overall objective of our proposal.

1) *Supply chain awareness*: We achieve consideration of the supply chain through a mechanism for MP parameter maintenance. The idea is to mark parameters as *required* when they are parameters of a failure model that can be applied on a lower tier. This process refers to an ontology-based repository of failure models which is part of the proposed system.

2) *Multitude of transformation approaches*: To distinguish between transformation approaches we adopt the classification of Czarnecki and Helsen in [13]. To fulfill evaluation criterion **EC.Transform** our proposed system includes a *direct-manipulation* as well as a *relational* M2M transformation approach.

3) *Extensibility*: Our concept implements extensibility by providing a plug-in mechanism. The concrete design of this mechanism depends on the chosen technology with which the system is implemented. In our case for example, we used a multi-paradigmatic (object-oriented and functional) programming language to implement the transformation system. More transformation approaches can be added to the system by implementing them using predefined interfaces.

4) *Modifiability*: Our approach considers modifiability of the behavior of transformation approaches in the design of the implemented transformation system. This is primarily achieved by the fact that the specification of a transformation according to the chosen approach already implies a definition of the behavior. In addition, transformation approaches can be parameterized, which has the positive side effect of encouraging reuse.

5) *Unvarying performance*: To handle different levels of detail in MPs the system relies only on established libraries which are known to perform well. Of course concrete definitions of transformations have a direct impact on performance. Therefore, it is mostly up to the user to keep that in mind while defining transformations.

C. Framework design

This section describes a framework for the lifting of MP documents to ontologies. We utilized existing tooling of the *Semantic Mission Profile Aware Design Platform* to map MP documents to corresponding ontologies. The platform contains a tool which takes a MP document in MPFO document format as input and generates a corresponding OWL ontology which is serialized in RDF/XML format.

1) *Architecture and implementation*: The platform is implemented as a Play Framework⁴ application using the Scala [14] programming language. This framework is based on the RESTful [15] architectural design pattern. The idea behind the design of the platform was on the one hand to provide means to create applications supporting MPAD and on the other hand to accelerate development by providing a pragmatic way to actually implement such applications.

2) *Core functionality*: The primary purpose of the Semantic MPAD Platform is the processing, interlinking, and integration of MPs and requirements. All inputs which are expected to be documents are lifted to ontologies by mapping engines. The platform moreover contains tools to detect and define actions upon changes to these documents. Using the standardized ontology language OWL as output opens up the possibility to easily integrate reasoning mechanisms into the processing of MPs and requirements.

3) *Interfaces*: Apart from the RESTful API (see section IV-C1) which can be accessed from any application capable of communicating via HTTP, the platform provides a web-based GUI using HTML and JavaScript. This allows interaction with the system from any standard-conforming browser and therefore eases the integration of the system into existing design processes and environments.

D. Architecture

This section describes the failure model and transformation ontology as well as the architecture of the suggested MP transformation system.

1) *Failure model and transformation ontology*: To consider the supply chain and reach evaluation criterion EC.Parameter an ontology addressing the parameter maintenance is introduced. This ontology has the purpose of identifying parameters which need to be maintained through a transformation. The OWL ontology contains SWRL rules for the inference of parameters to maintain. We explain the reasoning utilizing the combination of OWL axioms and SWRL rules by highlighting key axioms and rules of the ontology.

FailureModel The concept of a failure model is modeled by a subclass of the class expressed by axiom 1:

$$\begin{aligned} &\exists \text{hasEquation.xsd:string} \sqcap \\ &\exists \text{hasMissionProfileParameter.MissionProfileParameter} \sqcap \quad (1) \\ &\exists \text{offFailure.Failure} \end{aligned}$$

TechnologicalFailureModel A technological failure model is a specialization and therefore a subclass of a failure model. Moreover, it is also a subclass of the class expressed by axiom 2:

$$\begin{aligned} &\exists \text{hasProductParameter.ProductParameter} \sqcap \\ &\exists \text{hasTechnologyParameter.TechnologyParameter} \sqcap \quad (2) \\ &\exists \text{offFailure.TechnologicalFailure} \sqcap \\ &= 1 \text{ hasLayer.Layer} \end{aligned}$$

MPOTransformation A transformation is modeled as a class which is also a subclass of the class expressed by axiom 3:

$$\begin{aligned} &\geq \text{maintainsParameter.MissionProfileParameter} \sqcap \\ &= 1 \text{ ofSupplyChainLevel.SupplyChainLevel} \quad (3) \end{aligned}$$

The reasoning moreover utilizes SWRL rules to mark parameters which are to be maintained through a transformation. This will be described in the following.

MaintainParametersOEM Rule 4 sets MP parameters which need to be maintained on a MPOTransformation instance of OEMLevel when there is a TechnologicalFailureModel instance which uses these parameters. Depending on the supply chain, rules must be defined analogously for each tier of the supply chain.

$$\begin{aligned} &\text{MPOTransformation(trans)} \wedge \\ &\text{ofSupplyChainLevel(trans, OEMLevel)} \wedge \\ &\text{TechnologicalFailureModel(fm)} \wedge \quad (4) \\ &\text{hasMissionProfileParameter(fm, param)} \\ &\rightarrow \text{maintainsParameter(trans, param)} \end{aligned}$$

MaintainParametersHighLevel We also need to mark all parameters which need to be maintained with regard to high level failure models. Rule 5 shows that this can be achieved by checking whether a failure model has a root failure residing on the technology level.

$$\begin{aligned} &\text{MPOTransformation(trans)} \wedge \\ &\text{ofSupplyChainLevel(trans, OEMLevel)} \wedge \\ &\text{HighLevelFailureModel(fm)} \wedge \\ &\text{offFailure(fm, failure)} \wedge \quad (5) \\ &\text{hasRootFailure(failure, rootFailure)} \wedge \\ &\text{TechnologicalFailure(rootFailure)} \wedge \\ &\text{hasMissionProfileParameter(fm, param)} \\ &\rightarrow \text{maintainsParameter(trans, param)} \end{aligned}$$

Furthermore, a taxonomy for failure models was extracted from the categories described by Maricau and Gielen in [16]. Complementary metal-oxide-semiconductor (CMOS) device failure models for Electromigration (EM), Hot-carrier injection (HCI), Time-dependent dielectric breakdown (TDDB) and Bias temperature instability (BTI) were added as instances of corresponding classes. Fig. 4 shows the taxonomy of the failure model and transformation ontology.

⁴<https://www.playframework.com/>

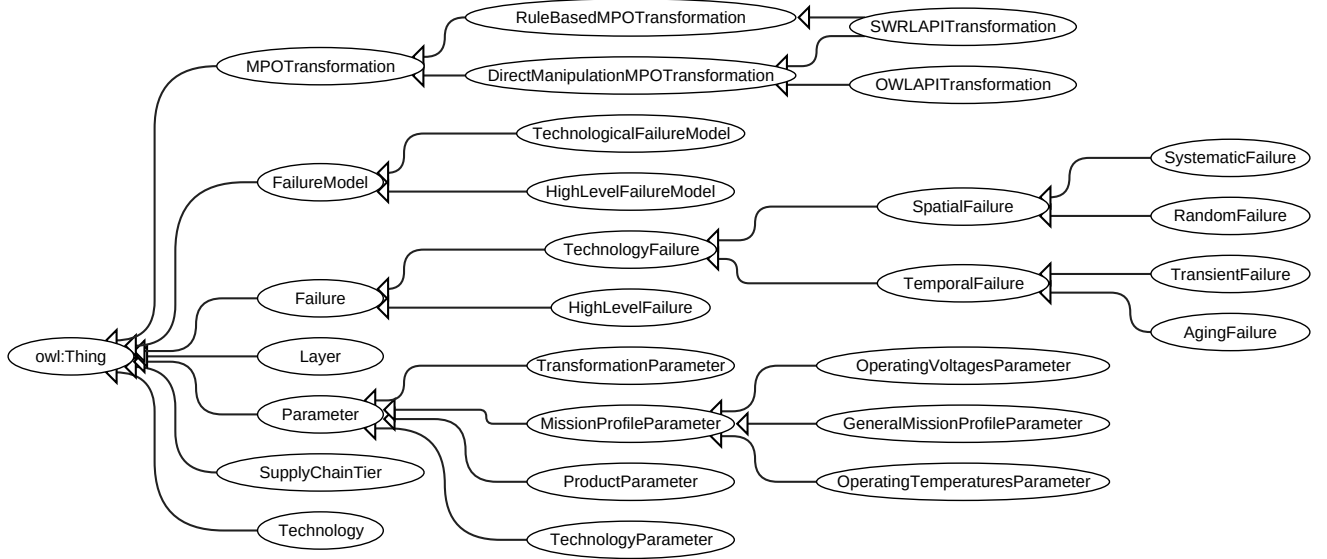


Fig. 4: Taxonomy of the failure model and transformation ontology

2) *System*: Please refer to Fig. 5 for an overview of the type hierarchy. Please note that we used the Scala programming language which provides so-called *traits* as opposed to interfaces from object-oriented languages such as Java. Scala traits allow multiple inheritance and can have attributes. UML does not have a semantic concept for this type of inheritance. Each object in the system is a *TransformationObject* that provides a *metadata* attribute of arbitrary type, based on key-value access. To meet evaluation criteria `EC.Transform` and `EC.Extension`, we introduced a trait for transformations: *MPOTransformation*. This trait holds *Mappings* which themselves have *sources* and *targets* lists attributes. As an example we have included a *OneToOneMapping* which extends the *Mapping* trait and overrides the *sources* and *targets* attributes to provide lists containing only a single element. Designing this way allows the definition of one-to-one, one-to-many, many-to-one and many-to-many mappings. This is required to create the necessary flexibility in MP transformation to cover use cases such as for example merging of several MPs into a single one. Actual data such as file paths, URLs or database addresses needed to use the sources respectively targets can be transported via the metadata.

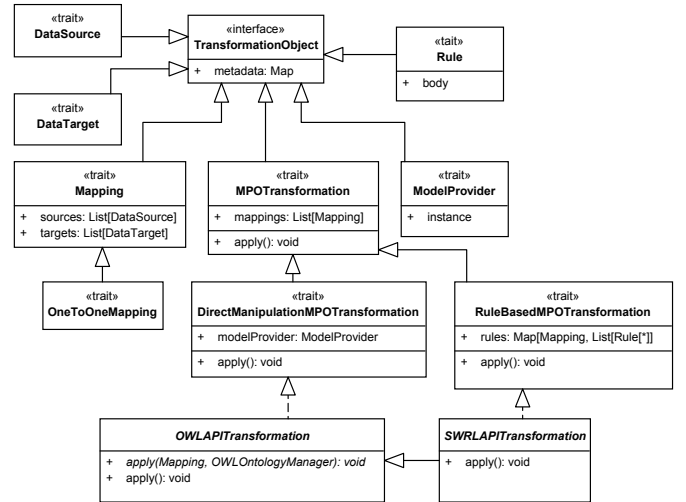


Fig. 5: UML diagram of the proposed system

A *DirectManipulationMPOTransformation* object has a *modelProvider* attribute which is used to access an object providing the model manipulation API as demanded by direct manipulation transformation approaches. *RuleBasedMPOTransformation* objects contain a *rules* attribute which is basically a mapping of *Mapping* objects to a list of *Rule* objects. This allows specific association of rules to certain mappings. A *Rule* object itself holds a *body* attribute of arbitrary type. While the trait *OWLAPITransformation* extends *DirectManipulationMPOTransformation*, a *SWRLAPITransformation* extends the latter trait as well as *RuleBasedMPOTransformation*.

To satisfy evaluation criterion `EC.Modify`, the system lets users define transformation behavior via overriding the *apply* method of transformation traits in implementing classes or specializing traits.

V. USE CASES

Two typical industrial use cases are described in this section which are concerned with the application of technology failure models utilizing the parameter maintenance mechanism and the derivation of sub-component MPs. For illustration purposes, both use case descriptions refer to concrete examples.

A. Technology failure model application

To apply a failure model at the technology level, certain MP parameters need to be maintained in a transformation. We used a MP document in MPFO version 0.2.1 as an example which contained seven parameters ($t_0, t_1, t_2, v_0, v_1, v_2, failureRate$) and temperature ranges for stress testing a component expressed as eight intervals ranging from -10 °C to 90 °C, see Tab. I. Prior to actually applying the transformation to the MP, the failure model and transformation

Interval ID	Min value	Max value
Interval_14	-10.0	20.0
Interval_15	20.0	30.0
Interval_16	30.0	40.0
Interval_17	40.0	50.0
Interval_18	50.0	60.0
Interval_19	60.0	70.0
Interval_20	70.0	80.0
Interval_21	80.0	90.0

Tab. I: MP temperature intervals

ontology is used to find out which parameters need to be maintained. This requires the user to specify the supply chain tier at which the transformation is to be performed. The existence of a knowledge base providing failure models is assumed. In our example, the ontology contains five front-end-of-line (FEOL) and back-end-of-line (BEOL) failure models *FEOL-TDDB*, *FEOL-HCI*, *FEOL-BTI*, *BEOL-TDDB* and *BEOL-EM* with corresponding technology, product and MP parameters defined. This set of MP parameters contained voltage and temperature parameters as well as the failure rate. Utilizing reasoning as described in section IV-D1, we found that temperature, voltage, and failure rate parameters needed to be maintained. List. 1 shows an explanation from the ontology editor Protégé [17] for the maintenance of an example temperature parameter.

```
writeError hasRootFailure HCI
DirectManipulationMPOTTransformation SubClassOf MPOTTransformation
example-transform-oem ofSupplyChainLevel oemLevel
example-transform-oem Type SWRLAPITransformation
failureModel-writeError hasMissionProfileParameter
exampleMissionProfileParameter_t1
SWRLAPITransformation SubClassOf DirectManipulationMPOTTransformation
HCI Type AgingFailure
AgingFailure SubClassOf TemporalFailure
failureModel-wirteError Type HighLevelFailureModel
MPOTTransformation(?trans), ofSupplyChainLevel(?trans, oemLevel),
HighLevelFailureModel(?fm), ofFailure(?fm, ?failure), hasRootFailure(?
failure, ?rootFailure), TechnologicalFailure(?rootFailure),
hasMissionProfileParameter(?fm, ?p) -> maintainsParameter(?trans, ?p)
failureModel-writeError ofFailure writeError
TemporalFailure SubClassOf TechnologicalFailure
```

List. 1: Protégé explanation for an OEM transformation which maintains an example temperature parameter

After identifying parameters which need to be maintained, the transformation operations are specified. In our example, we did this by implementing a subclass of the *OWLAPITransformation* trait, please see List. 2 which shows an excerpt of the transformation implementation. To increase the ranges, we implemented a transformation which decreased the lower and increased the upper limits of the intervals. To execute the transformation, the input MPFO document is mapped to an ontological representation. We used the framework described in section IV-C for this task.

Characteristic	Input ontology	Output ontology
Axioms	3874	3633
Logical axioms	2889	2773
Declaration axioms	911	860
Class count	52	36
Object properties	39	31
Data properties	30	19
Individuals	775	774
Object property assertions	779	779
Data property assertions	1216	1216

Tab. II: Ontology metrics of input and output MP ontologies

```
class IncreaseTemperatureCycleRange {
  sourcePath: String,
  targetPath: String,
  rangeIncrease: Float) extends OWLAPITransformation {
  override val metadata = Map("description" -> "temperature range increase")
  override val mappings = List(new OneToOneMapping(
    new DataSource { override val metadata = Map("path" -> sourcePath) },
    new DataTarget { override val metadata = Map("path" -> targetPath) })
  )
  override def apply(mapping: Mapping, manager: OWLOntologyManager) = {
    assert(mapping.sources.size == 1 && mapping.targets.size == 1)
    assert(mapping.sources(0).metadata.isDefinedAt("path"))
    assert(mapping.sources(0).metadata("path").isInstanceOf[String])
    assert(mapping.targets(0).metadata.isDefinedAt("path"))
    assert(mapping.targets(0).metadata("path").isInstanceOf[String])
    val source = manager.loadOntologyFromOntologyDocument(
      new File(mapping.sources(0).metadata("path").asInstanceOf[String]))
    val target = manager.createOntology(
      IRI.create("transformed"), Set(source.asJava)
    )
    manager.setOntologyDocumentIRI(target, IRI.create(
      new File(mapping.targets(0).metadata("path").asInstanceOf[String]))
    )
    manager.setOntologyFormat(target, manager.getOntologyFormat(source))

    // TRANSFORMATION OPERATIONS

    // Apply the changes and save the result
    manager.applyChanges(axiomsToRemove.map(ax =>
      new RemoveAxiom(target, ax).asJava)
    )
    manager.applyChanges(axiomsToAdd.map(ax =>
      new AddAxiom(target, ax).asJava)
    )
    manager.saveOntology(target)
  }
}
```

List. 2: Scala source code template for an OWLAPI-based transformation

Tab. II lists metrics of transformation input and output ontologies. Even though some axioms have been removed in the transformation, the object and data property assertions are preserved. The loss of some axioms is due to intentionally discarded MP information such as operating states and ports specifications.

B. Sub-component MP derivation

This use case describes the derivation of a MP for a sub-component from a base MP. As an example, we consider the development of an automotive Electronic Control Unit (ECU). A Tier-1 assembles the ECU module while a Tier-2 provides an Integrated Circuit (IC) component for the ECU which in turn is manufactured using technology provided by a Tier-3. MPs are used in the engineering of the ECU, the IC component and the underlying technology.

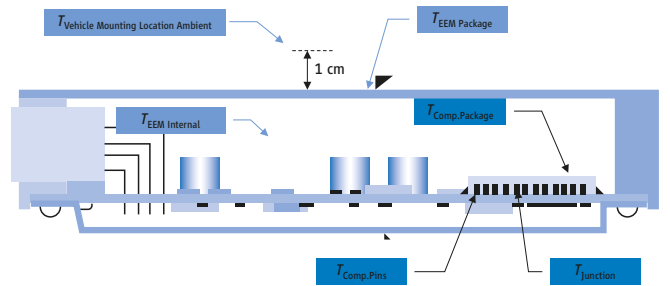


Fig. 6: Temperature measurement points of an ECU (here called Electrical/Electronic Module, EEM) defined in the Handbook for Robustness Validation, from [7]

In our example, the ECU is mounted in the engine compartment. Using a publicly available typical temperature load distribution [18] one can define a MP for the ECU. This temperature profile of the inner air temperature of the ECU (referring to Fig. 6: $T_{EEM\ Internal}$) is expressed as a histogram of 12 temperature ranges between -40°C and 125°C . For providing a MP to the Tier-3 the Tier-1 has to transform the temperature profile MP data.

The JEDEC standard JESD51-2 defines a methodology for performing thermal resistance measurement for ICs in natural convection. The thermal resistance between device junction and ambient air is determined using equation 6. In its second form in equation 7, given the ambient temperature T_A , the total device power dissipation P , and the total thermal resistance between device junction and the ambient air θ_{JA} , one can calculate the junction temperature T_J :

$$\theta_{JA} = (T_J - T_A)/P \quad \Leftrightarrow \quad (6)$$

$$T_J = T_A + (P \times \theta_{JA}) \quad \Leftrightarrow \quad (7)$$

$$= \theta_{JC} + \theta_{CA} \quad (8)$$

where θ_{JC} is the thermal resistance between junction and case of the device and θ_{CA} the thermal resistance between case of the device and ambient air. Equations 7 and 8 are also used in the industry for junction temperature calculation⁵. These equations are used to calculate a corresponding temperature profile for the Tier-3 based on the ECU MP.

Using our proposed system, a MP for the IC sub-component technology can be derived from the base ECU MP while MP parameters required for failure model application are maintained. The actual specification of the transformation operations requires expert knowledge about the specific relationships between the loads of the base component and the sub-component. These relationships are reflected in the transformation. The junction temperature calculation is an example for such relationships. Parameters which are not identified as required can be stripped off the base MP to protect IP and reduce required storage space. As MPs are required for each sub-component of the ECU the transformation operations specification can be reused thus saving time and avoiding errors.

Suppose one can assume that the failure model and transformation ontology contains (1) information about the supply chain structure and (2) corresponding failure models. In that case an exemplary flow to derive a MP for the IC sub-component under consideration of parameters for failure model application would be:

- 1) The user specifies the supply chain tier at which the transformation is performed in the failure model and transformation ontology for example by using an OWL ontology editor, see List. 3.

```

Individual: example-transform-oem

Types:
  SWRLAPITransformation

Facts:
  ofSupplyChainLevel oemLevel

```

List. 3: OWL Individual representing a OEM transformation

- 2) The system identifies parameters which need to be maintained using the failure model and transformation ontology and any OWL reasoner. Parameters to maintain will be shown as objects related to the transformation instance by the *maintainsParameter* property, see List. 4.

⁵For example in the tutorial MT-093 from Analog Devices, in the application report SPRA953C from Texas Instruments, the common information document TB379 from Renesas or the introduction to temperature specification document AN4017 from Cypress.

```

Individual: example-transform-oem

Types:
  OWLAPITransformation

Facts:
  ofSupplyChainLevel oemLevel
  maintainsParameter MPParam_failureRate
  maintainsParameter MPParam_v0
  maintainsParameter MPParam_v1
  maintainsParameter MPParam_v2
  maintainsParameter MPParam_t0
  maintainsParameter MPParam_t1
  maintainsParameter MPParam_t2

```

List. 4: OWL Individual representing a OEM level transformation in Manchester syntax. Three MP parameters are maintained which is the inferred knowledge

- 3) The user specifies the transformation operations via implementing a *MPOTransformation* trait or one of the extended traits. This includes setting data sources and targets of the transformation, see List. 2. Equations 7 and 8 are used to actually implement corresponding transformation operations.
- 4) The system maps the base MP of the ECU in MPFO format to an OWL ontology. This task is carried out using a component of the platform described in section IV-C.
- 5) The system finally executes the transformation which results in the IC MP ontology.

VI. CONCLUSION

We suggested a transformation system for requirements represented by MPs which is capable of considering the supply chain with respect to high level and technology failure models. The extensible design allows for the integration of multiple transformation approaches which was identified as an important requirement for a MP transformation system.

A limitation of the SWRL rule-based transformation is that SWRL supports monotonic inference only. Because of that SWRL-based transformations can only add more facts to an ontology but neither modify nor remove existing information. This could be mitigated for example by adding pre- and post-processing steps to SWRL transformations. In these extra processing steps facts modification or removal is handled. Another limiting factor which is related to the current implementation of the component which maps MPs to ontological representations, as described in section IV-C, is that the ontology model is constructed in memory. This is problematic for very large MPs which do not occur so frequently in practice.

There are also opportunities to improve the approach. General improvements could consist of approaches for reuse of M2M transformations and verification thereof [19, 20]. An improvement with regard to the set of supported transformation approaches could be integration of pattern-based transformation [21]. Further research can also be conducted on direct failure model evaluation using SWRL Built-In extensions.

We considered other technical solutions upfront such as using the Eclipse Modeling Framework (EMF) [22] instead of ontologies and model transformation languages such as the Epsilon Transformation Language (ETL) [23] or the Atlas Transformation Language (ATL) [24] but decided to keep the system design simple in order to reduce complexity. This turned out to be a viable way to realize a MP transformation system which confirms previous results of the literature [25]. We used standardized languages and formats which are in the process of being standardized and showed the practical applicability of our approach.

VII. ACKNOWLEDGEMENT

This paper is partially funded by the BMBF within the project SAFE4I (grant number 01|S17032C), and by the BMWi within the project InsightProducts (grant number 228EN/2).

REFERENCES

- [1] J. D'Ambrosio and G. Soremekun, "Systems engineering challenges and MBSE opportunities for automotive system design," in *2017 IEEE International Conference on Systems, Man, and Cybernetics (SMC)*, Oct 2017, pp. 2075–2080.
- [2] L. Wang, M. Izygon, S. Okon, H. Wagner, and L. Garner, "Effort to accelerate MBSE adoption and usage at JSC," in *AIAA SPACE 2016*, 2016, p. 5542.
- [3] W3C OWL Working Group, "OWL 2 Web Ontology Language Document Overview (Second Edition) - W3C Recommendation 11 December 2012," 2012.
- [4] M. O'Connor, H. Knublauch, S. Tu, B. Grosz, M. Dean, W. Grosso, and M. Musen, "Supporting rule system interoperability on the semantic web with swrl," in *International semantic web conference*. Springer, 2005, pp. 974–986.
- [5] C. Bizer, T. Heath, and T. Berners-Lee, "Linked Data - the story so far," *Semantic Services, Interoperability and Web Applications: Emerging Concepts*, pp. 205–227, 2009.
- [6] O. Chourabi, Y. Pollet, and M. B. Ahmed, "Ontology based knowledge modeling for system engineering projects," in *Second International Conference on Research Challenges in Information Science, 2008. RCIS 2008*. IEEE, 2008, pp. 453–458.
- [7] C. Byrne and Zentralverband Elektrotechnik und Elektronikindustrie, *Handbook for robustness validation of automotive electrical/electronic modules*. ZVEI, 2008.
- [8] T. Nirmaier, A. Burger, G. Harrant, A. Viehl, O. Bringmann, W. Rosenstiel, and G. Pelz, "Mission profile aware robustness assessment of automotive power devices," in *Design, Automation and Test in Europe Conference and Exhibition (DATE), 2014*, 2014, pp. 1–6.
- [9] U. Abelein, H. Lochner, D. Hahn, and S. Straube, "Complexity, quality and robustness - the challenges of tomorrow's automotive electronics," in *Design, Automation & Test in Europe Conference & Exhibition (DATE), 2012*. IEEE, 2012, pp. 870–871.
- [10] G. Jerke and A. B. Kahng, "Mission profile aware IC design: a case study," in *Proceedings of the conference on Design, Automation & Test in Europe*. European Design and Automation Association, 2014, p. 64.
- [11] R. Rongen, "Knowledge based qualification of semiconductor devices - introduction to the NXP Reliability Knowledge Framework," in *European CEEES Seminar*, 2014.
- [12] N. Kahani, M. Bagherzadeh, J. R. Cordy, J. Dingel, and D. Varró, "Survey and classification of model transformation tools," *Software & Systems Modeling*, pp. 1–37, 2018.
- [13] K. Czarnecki and S. Helsen, "Classification of model transformation approaches," in *Proceedings of the 2nd OOPSLA Workshop on Generative Techniques in the Context of the Model Driven Architecture*, vol. 45, no. 3. USA, 2003, pp. 1–17.
- [14] M. Odersky, P. Altherr, V. Cremet, B. Emir, S. Maneth, S. Micheloud, N. Mihaylov, M. Schinz, E. Stenman, and M. Zenger, "An overview of the Scala programming language," Tech. Rep., 2004.
- [15] R. T. Fielding and R. N. Taylor, *Architectural styles and the design of network-based software architectures*. University of California, Irvine Doctoral dissertation, 2000.
- [16] E. Marica and G. Gielen, "CMOS reliability overview," in *Analog IC Reliability in Nanometer CMOS*. Springer, 2013, pp. 15–35.
- [17] M. A. Musen *et al.*, "The protégé project: a look back and a look forward," *AI matters*, vol. 1, no. 4, p. 4, 2015.
- [18] T. Hogenmüller and H. Zinner, "Tutorial for Lifetime Requirements and Physical Testing of Automotive Electronic Control Units (ECUs)," Jul. 2012.
- [19] A. Kusel, J. Schönböck, M. Wimmer, G. Kappel, W. Retschitzegger, and W. Schwinger, "Reuse in model-to-model transformation languages: are we there yet?" *Software & Systems Modeling*, vol. 14, no. 2, pp. 537–572, 2015.
- [20] L. A. Rahim and J. Whittle, "A survey of approaches for verifying model transformations," *Software & Systems Modeling*, vol. 14, no. 2, pp. 1003–1028, 2015.
- [21] D. Wagelaar, R. Van Der Straeten, and D. Derudder, "Module superimposition: a composition technique for rule-based model transformation languages," *Software & Systems Modeling*, vol. 9, no. 3, pp. 285–309, 2010.
- [22] D. Steinberg, F. Budinsky, E. Merks, and M. Paternostro, *EMF: eclipse modeling framework*. Pearson Education, 2008.
- [23] D. S. Kolovos, R. F. Paige, and F. A. Polack, "The epsilon transformation language," in *International Conference on Theory and Practice of Model Transformations*. Springer, 2008, pp. 46–60.
- [24] F. Jouault, F. Allilaire, J. Bézivin, I. Kurtev, and P. Valduriez, "Atl: a qvt-like transformation language," in *Companion to the 21st ACM SIGPLAN symposium on Object-oriented programming systems, languages, and applications*. ACM, 2006, pp. 719–720.
- [25] D. H. Akehurst, B. Bordbar, M. J. Evans, W. G. J. Howells, and K. D. McDonald-Maier, "Sitra: Simple transformations in java," in *International Conference on Model Driven Engineering Languages and Systems*. Springer, 2006, pp. 351–364.